Session 24 Homework — Multi-Criteria Filter + Sort, and Closing a Real Disclosed Gap

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context (required)

By the end of class, Project Detail's own real issue list already filters by `status`, through a real Client Component reading and writing the real URL. This homework extends the SAME real pattern to a second real backend filter (`title`), adds a genuinely different kind of filtering (client-derived sort, since the real backend has no sort parameter at all), and closes a real, disclosed gap this session's own TEAMWORK left open on purpose.

Goal (required)

A real combined filter+sort feature on Project Detail: the real backend's own `status` / `title` query filters, sent together, plus a real client-derived sort over whatever the backend already filtered (CORE). A real, reproduced written answer about why this session pins Tailwind to v3 (STRETCH, carried from LC-01's own real finding). A real fix for TEAMWORK's own disclosed theme-persistence gap (CHALLENGE).

What you already have

A real, working `StatusFilter` Client Component (reads/writes the real URL's `status` parameter); a real `Button` / `Card` / `StatusBadge` set of reusable primitives; a real `ThemeProvider` / `ThemeToggle` (TEAMWORK) that works but does not yet survive a page reload — disclosed, not hidden, in TEAMWORK's own real `EXPECTED RESULT`.

What to build (required)

A real `TitleSearch` Client Component (the identical real URL-driven pattern as `StatusFilter`, for a free-text field instead of four fixed choices); a real `SortableIssueList` Client Component (a real controlled sort-order `<select>`, sorting the already-filtered issues with plain client state — never sent to the backend, since no backend sort parameter exists); Project Detail wired to use both together.

CORE (required)

How to start

Build `TitleSearch` first, as its own skeleton (read the real `title` URL parameter, render a plain `<input>` with no `onChange` yet) — the identical real discipline `StatusFilter` already used live. Prove it compiles before adding the real write-back logic.

Hints

- **Hint 1 (conceptual):** `TitleSearch`'s own real input must be UNCONTROLLED (`defaultValue`, not `value`) — think about why a controlled input would need to re-render on every real keystroke just to stay in sync with itself.
- **Hint 2 (where to look):** `StatusFilter.tsx`'s own real `setStatus` function — the exact same shape works for `title`, with one real difference: `title` comes from free text, not one of four fixed buttons.
- **Hint 3 (where to look):** the real backend's own `FindIssuesQueryDto` — confirm `title` is already a real, validated query parameter that may be left out of the request entirely, before wiring anything to it.

STRETCH (required)

How to start

Actually reproduce LC-01's own real finding yourself, in a throwaway scratch copy of `web/` — install Tailwind v4, try the real v4 config syntax, run a real build, and read the real error.

Hints

- **Hint 1 (conceptual):** the real error names a specific Node.js module-loading mechanism (`require()`) and a specific module format (`.mjs`, an ES module) — that pairing is the whole real story.
- **Hint 2 (where to look):** this program's own other real version pins (Next.js, Node itself) — the same real principle applies here.

CHALLENGE (required)

How to start

Re-read TEAMWORK's own `ThemeProvider` code and its own disclosed limitation, out loud, in `TW-04`'s real `EXPECTED RESULT`. Decide where a real stored choice should be checked, and in which of the two real `useEffect` calls.

Hints

- **Hint 1 (conceptual):** a real stored choice, when one exists, must win over the real OS preference — never the other way around.
- **Hint 2 (where to look):** the real browser's own `localStorage` API — `getItem` / `setItem`, nothing more exotic is needed.

What to research

1. Which of the two real `useEffect` calls in `ThemeProvider` should read `localStorage`, and which should write to it? Why not the same one for both?
2. What real value should be stored — the same `"light"` / `"dark"` string this file already uses everywhere else, or something else? Justify your real choice.

Rules (required)

Allowed: everything already shown LIVE/TEAMWORK this session — Tailwind utility classes, the real `Button` / `Card` / `StatusBadge` primitives, the real URL-driven Client Component pattern, React Context, `localStorage`.

Not yet: no Server Actions, no Redux Toolkit, no real authentication — all three arrive starting Session 25/26. No backend sort endpoint — the real backend genuinely has none; do not invent one.

Test it (required)

inside api/

BASH

```
PORT=3000 npm run start
```

inside web/ (second terminal)

BASH

```
npm run dev
```

in a third terminal

BASH

```
curl -s "http://localhost:3000/issues?projectId=1&status=todo&title=login"
npx tsc --noEmit
npm run build
```

Then, in a real browser: combine the real status filter, the real title search, and the real sort order in every combination you can think of; toggle the real theme, reload the page, confirm it survives.

Common confusions

- Filtering by title client-side (in JavaScript, over an already-fetched list) instead of sending the real backend's own `title` query parameter — the real backend already does this; reinventing it client-side contradicts this session's own entire lesson.
- Sorting on the backend, or inventing a fake backend sort parameter — the real backend genuinely has none; the whole point of `SortableIssueList` is doing this correctly, client-side.
- Storing the theme choice as a boolean instead of the real `"light"` / `"dark"` string already used everywhere else in `ThemeContext.tsx`.

Done when (required)

- Combining the real status filter and the real title search produces the exact real issues matching BOTH.
- Changing the real sort order re-sorts the currently-filtered list, client-side, instantly.
- `npx tsc --noEmit` and `npm run build` both exit clean.

- Reloading the page after toggling the theme keeps the real chosen theme, not the OS default.
- Your written STRETCH answer quotes the real, exact Tailwind v4 build error you reproduced.

Working time (required)

About 74 minutes total (HW-01…09) — CORE (filter+sort, HW-01…07) is ~52 minutes; STRETCH (the real Tailwind version investigation, HW-08) is ~10 minutes; CHALLENGE (theme persistence, HW-09) is ~12 minutes.